Campus Event Management & Feedback Analyzer

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410046	26102000168	BWU/MCA/26/051	Subhajit Maity

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



ABSTRACT

This project presents a Python-based application that manages campus events, student registrations, and analyzes feedback data from CSV files. The system reads event records, registration data, and feedback scores, calculates key metrics such as average ratings, event popularity, registration counts, and generates visual charts for better insights. It is developed using modular Python scripts (mainly main.py with supporting data files) and relies on libraries such as pandas, matplotlib, and related data-processing tools.

A simple command-line interface allows users to load data, manage events, register students, collect and analyze feedback, view statistics, and generate graphical reports without complex coding. The system identifies popular events, low-rated events, computes overall class and event statistics, and produces clear visualizations. This reduces manual calculation effort for event organizers and administrators and demonstrates how Python data analysis and visualization techniques can be combined into a practical campus management tool.

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	1
2	Objective	2
3	Planning	3
4	Requirement Analysis	5
5	System Flow	6
6	Proposed Design	13
7	Experimental Result	15
8	Future Scope	18
9	Conclusion	19

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	Context Level Diagram	6
5.2	Data Flow Diagram Level 0	7
5.3	Data Flow Diagram Level 1	8
5.4	Use Case Diagram	9
5.5	Activity Diagram	10
5.6	Sequence Diagram	11
5.7	Flow Chart	12
7.1	Data Loading	15
7.2	Event Management	15
7.3	Feedback Analysis	16
7.4	Bar Chart Generation	16
7.5	Pie Chart Generation	17
7.6	Histogram Generation	17

LIST OF SYMBOLS, ABBREVIATIONS, AND NOMENCLATURE

Symbol / Abbreviation	Description
CSV	Comma Separated Values (file format used for event & feedback data input)
DFD	Data Flow Diagram
CLD	Context Level Diagram
UML	Unified Modeling Language
GUI	Graphical User Interface
PDF	Portable Document Format
%	Percentage (used for Attendance / Rating Percentage)
Avg	Average (used for Average Rating / Feedback Score)
ID	Identifier (Event ID / Feedback ID)
→	Data flow / Message direction (in DFD and Sequence Diagram)
○ / ▭	Process / Activity symbol (in Flow Chart and Activity Diagram)
◇	Decision symbol (in Flow Chart and Activity Diagram)
▭ (open)	Data Store (in DFD)
Actor	External entity / User (in Use Case and Context diagrams)

CHAPTER 1: INTRODUCTION

Traditional campus event management and feedback collection relies on manual registers, spreadsheets, and time-consuming calculations. Event organizers often struggle to quickly identify popular events, low-rated sessions, or registration trends, and generating summary reports or visual insights requires additional effort. With the availability of Python data analysis libraries, it is now possible to automate large parts of this process.

This project focuses on building a system that reads event data, student registration records, and feedback scores from CSV files, processes the records, calculates important metrics, and produces both textual statistics and graphical charts. The idea is simple: the user provides structured data files, and the program computes averages, popularity rankings, registration counts, rating distributions, and class-level or event-level summaries, then visualizes the results.

The system uses a modular design with clear separation of data loading, analysis logic, chart generation, and the main program entry point. Behind the interface, the workflow follows clear steps: load data, validate records, compute performance and feedback metrics, generate visualizations, and present a consolidated report. The project demonstrates how data processing, statistical calculation, and visualization can be combined into a lightweight, easy-to-understand campus event analytics tool suitable for students and beginners.

CHAPTER 2: OBJECTIVE

The objective of this project is to develop a Python-based Campus Event Management & Feedback Analyzer that enables users to process event records, student registrations, and feedback data from CSV files and obtain meaningful insights. The system aims to provide a modular, automated workflow by integrating data loading, metric calculation, and visualization components. Users can load event and registration data, compute individual and overall statistics (average ratings, popularity scores, registration counts), identify high-performing or low-rated events, and generate charts without requiring advanced programming knowledge. By combining pandas for data handling, custom analysis functions, and matplotlib-based visualizations, the project intends to simplify event monitoring, reduce manual calculation effort, and offer an accessible end-to-end event management and feedback analysis pipeline.

CHAPTER 3: PLANNING

The planning phase outlines the step-by-step approach used to design, organize, and implement the Campus Event Management & Feedback Analyzer. Proper planning ensures that the workflow remains efficient, modular, and easy for beginners to understand and use.

1. Requirement Identification

- Understand the need for a simple Python tool to manage campus events, registrations, and analyze feedback from CSV data.
- Identify the target users (event organizers, students, academic administrators, beginners).
- Define the core requirement: CSV data → analysis of events, registrations & feedback → statistics + charts.

2. System Design Planning

- Decide to use pandas for data handling and matplotlib for visualization.
- Choose a modular structure with clear functions for data loading, event management, registration handling, feedback analysis, and chart generation.
- Plan the workflow from data loading to final report and charts.
- Select supporting libraries listed in requirements (pandas, matplotlib).

3. Pipeline Structure

Design a sequential pipeline with the following steps:

- Data loading from events.csv, registrations.csv, students.csv
- Event and registration object creation / data validation
- Feedback metric calculation (average rating, distribution)
- Event popularity and registration statistics
- Identification of top and low-rated events
- Statistical summaries
- Chart generation
- Result display

4. File & Module Planning

- main.py – Entry point, menu, and core logic
- events.csv – Sample event data
- registrations.csv – Sample registration data
- students.csv – Sample student data
- requirements (implied) – Dependencies (pandas, matplotlib)

5. Code Structure Planning

- Break the code into clear, reusable sections:
- Data loading and validation

- Event management functions
- Registration and feedback handling
- Metric calculation and filtering of low-rated events
- Chart plotting functions
- Main menu / command-line interface

6. Testing & Validation Plan

- Test with sample CSV data.
- Verify correct average rating, popularity, and registration calculations.
- Check chart generation for different metrics.
- Validate handling of edge cases (missing data, zero registrations, etc.).

7. Scalability & Improvement Planning

Plan for expandable features such as:

- Support for multiple campuses or departments
- Export of reports to PDF or Excel
- Interactive GUI (Tkinter or Streamlit)
- Predictive analysis using simple machine learning
- Database integration instead of CSV

CHAPTER 4: REQUIREMENT ANALYSIS

Functional Requirements

1. Data Input

- The system must read event, registration, and student records from CSV files (events.csv, registrations.csv, students.csv).
- Each record typically includes event identifiers, dates, categories, student details, and feedback scores.
- Input is handled through Python file reading and pandas DataFrame operations.

2. User Interaction

- A simple command-line interface or menu-driven program in main.py.
- Options to load data, manage events, register students, collect feedback, view statistics, generate charts, and exit.
- Users should not need to write code to obtain results.

3. Model and Framework Requirements

The project depends on the following Python libraries:

- pandas (data loading and manipulation)
- matplotlib (chart generation)
- Other standard libraries (os, csv, etc.) as needed

4. Analysis Requirements

For every event and feedback record:

- Calculate average feedback rating / total performance score.
- Calculate registration counts and popularity percentages.
- Identify high-rated and low-rated events.
- Determine overall event statistics and trends.

5. Visualization Requirement

- Generate bar charts, pie charts, or other suitable plots for rating distribution, event popularity, registration comparison, etc.
- Charts should be saved or displayed clearly.

6. Output Organization

The system must:

- Display textual summaries and statistics on the console.
- Generate and save chart images.
- Provide a clear final report of event-level and overall insights.

CHAPTER 5: SYSTEM FLOW

Context Level Diagram (CLD):

It shows the overall system as a single process interacting with external entities (User and CSV data sources) and producing reports/charts.

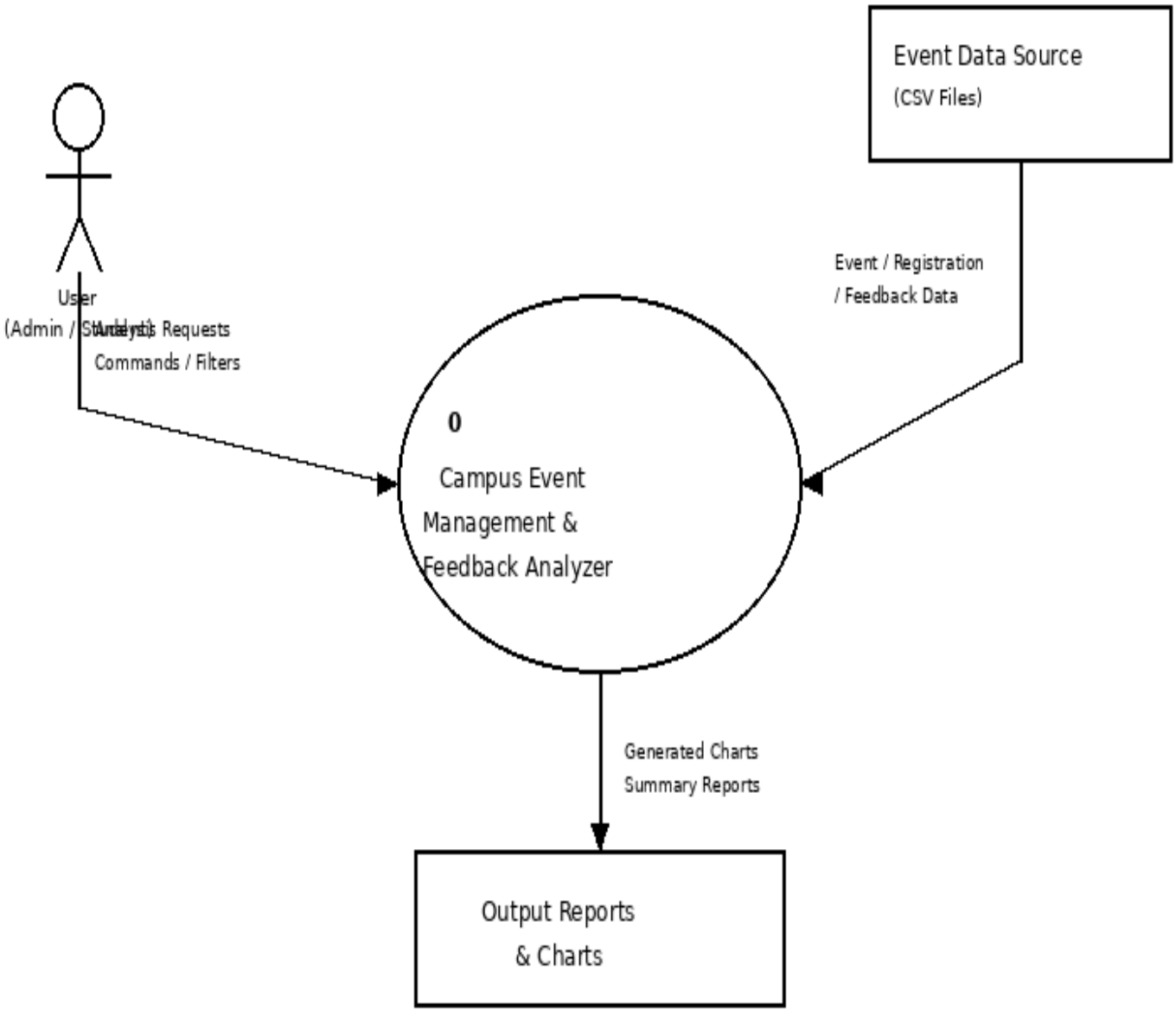


Fig 5.1: Context Level Diagram

Data Flow Diagrams (DFD):

It illustrates the major processes (Load Data, Manage Events, Handle Registrations, Analyze Feedback, Generate Statistics, Create Charts) along with data stores and high-level data flow.

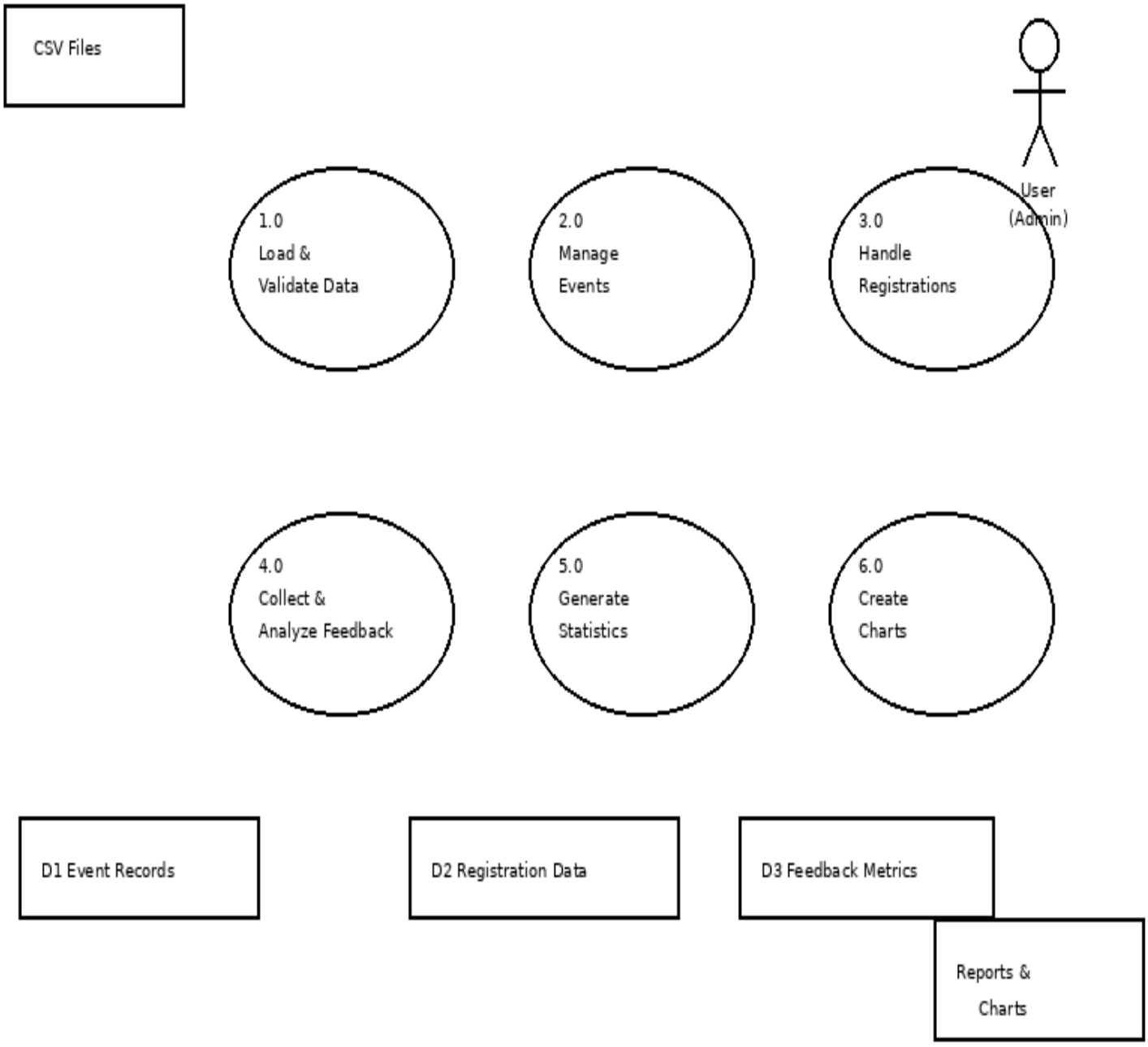


Fig 5.2: Data Flow Diagram Level 0

Data Flow Diagrams (DFD):

It breaks down the Level 0 processes into detailed sub-processes for data loading, event management, registration handling, feedback analysis, statistics generation, and chart creation.

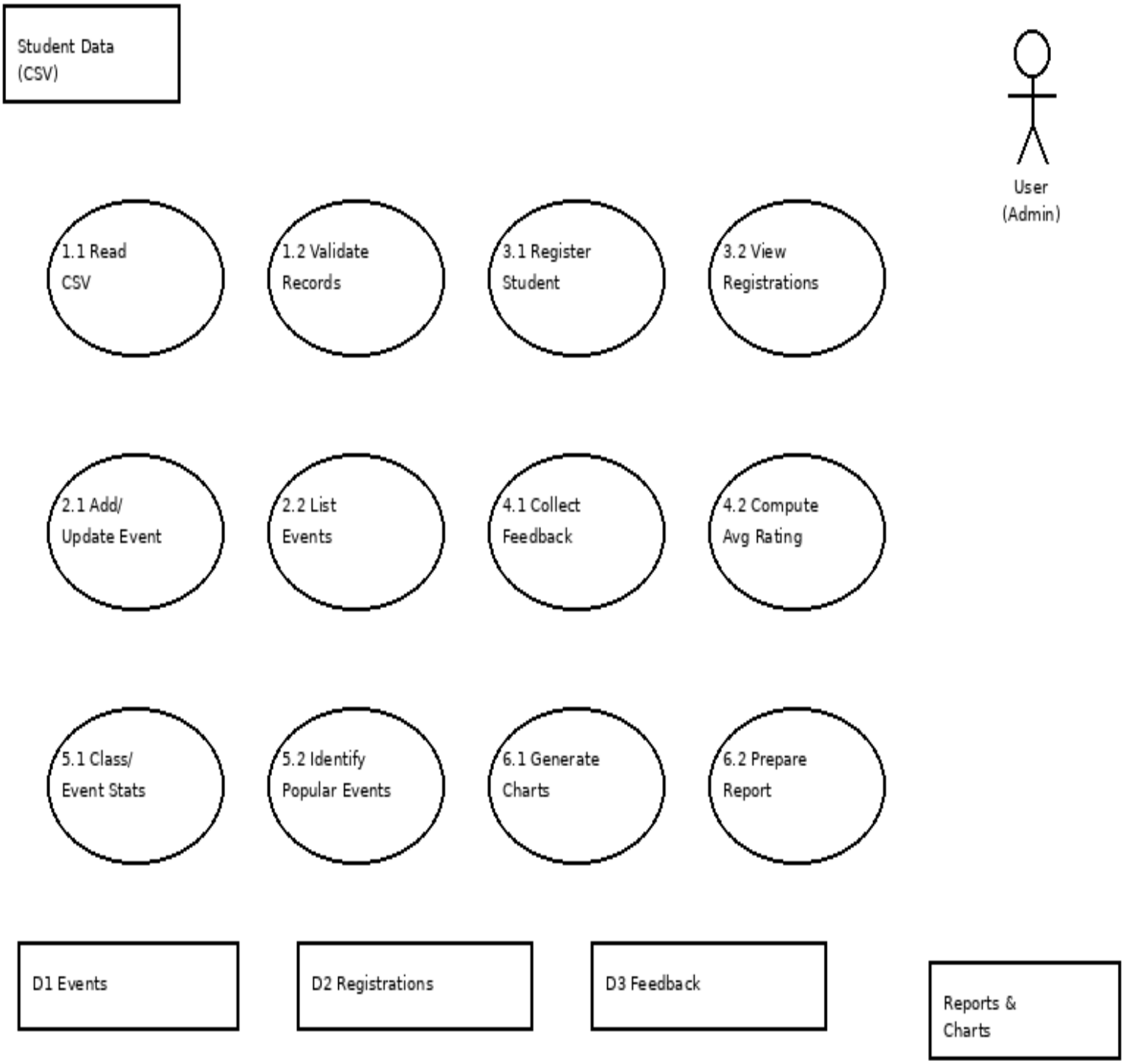


Fig 5.3: Data Flow Diagram Level 1

Use Case Diagram:

It depicts the interactions between the Admin/Student and the system's key functionalities such as loading data, managing events, registering students, analyzing feedback, and generating reports.

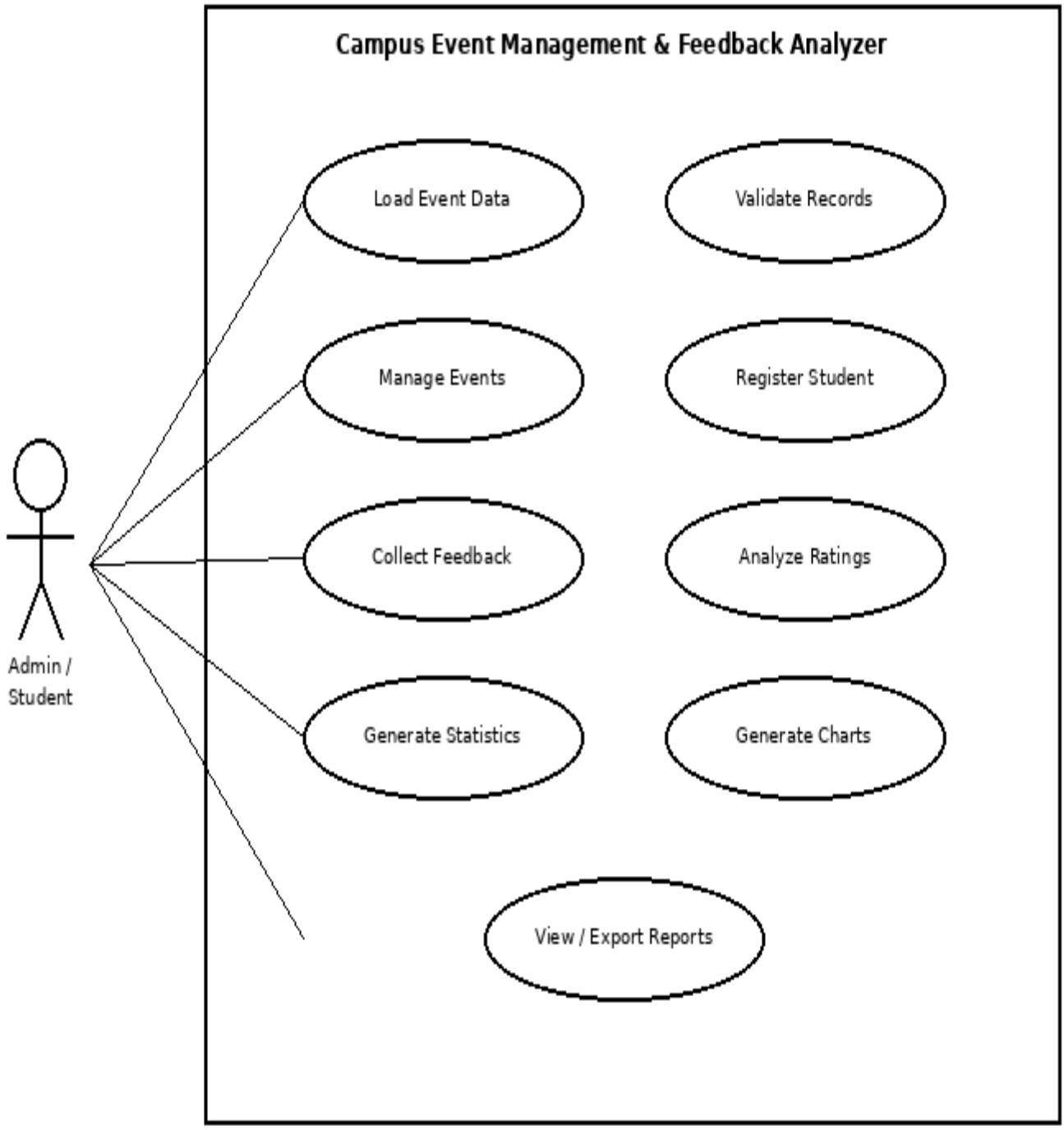


Fig 5.4: Use Case Diagram

Activity Diagram:

It represents the step-by-step workflow of the system from starting the application to displaying the final results, including decision points for data validation.

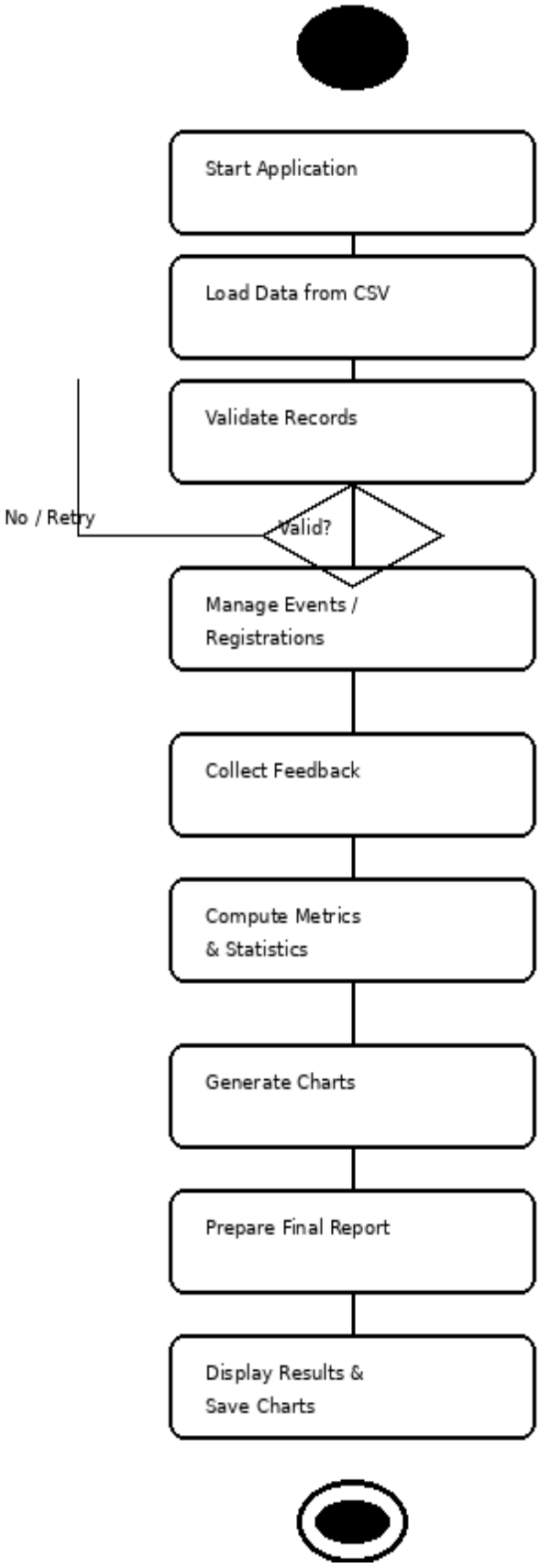


Fig 5.5: Activity Diagram

Sequence Diagram:

It shows the chronological sequence of messages exchanged between the User, Main Controller, Data Loader, Analysis Module, Chart Generator, and Report Output.

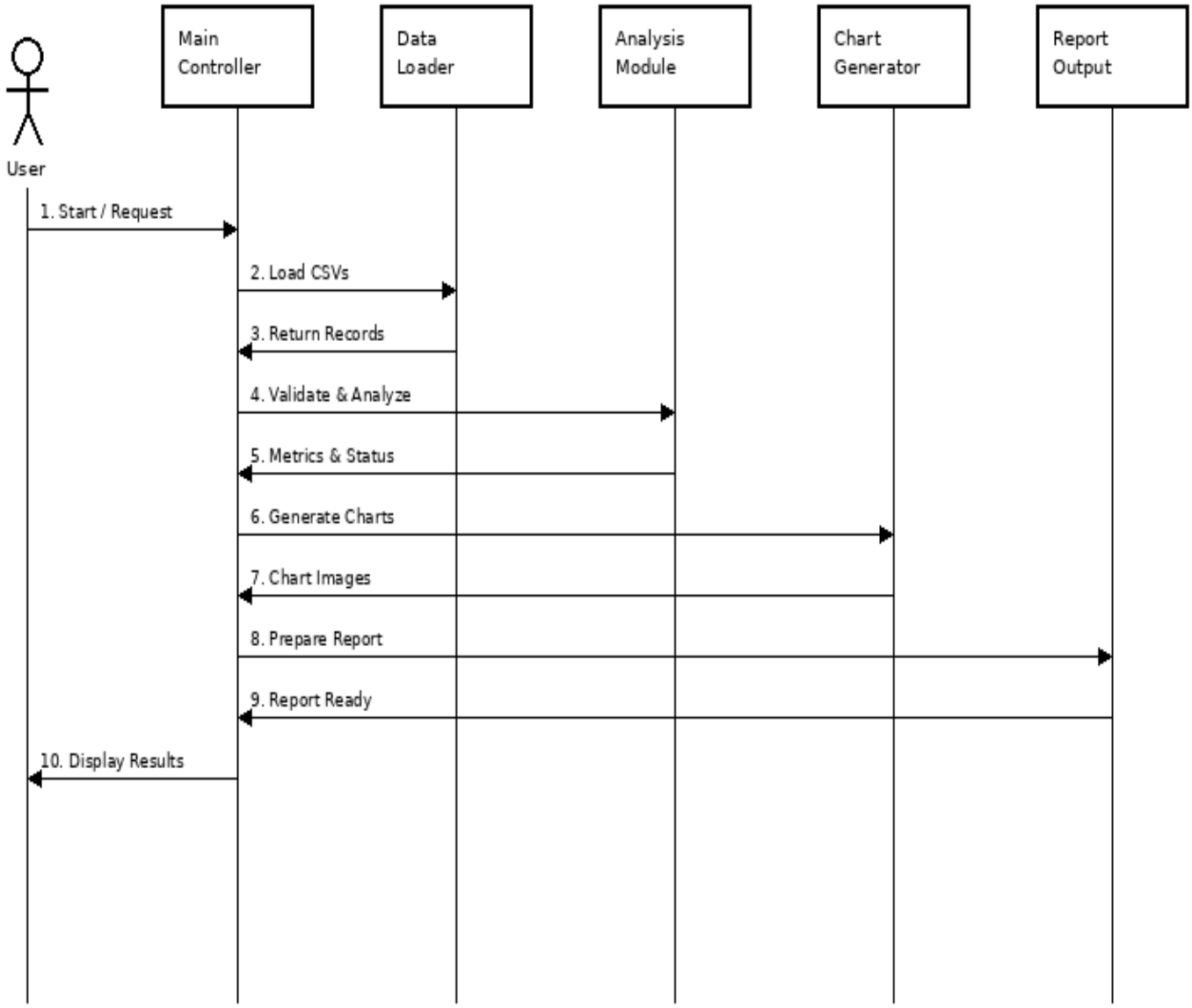


Fig 5.6: Sequence Diagram

Flow Chart:

It presents the complete sequential logic of the system using standard flowchart symbols, including decision points for data validation and the path to final report generation.

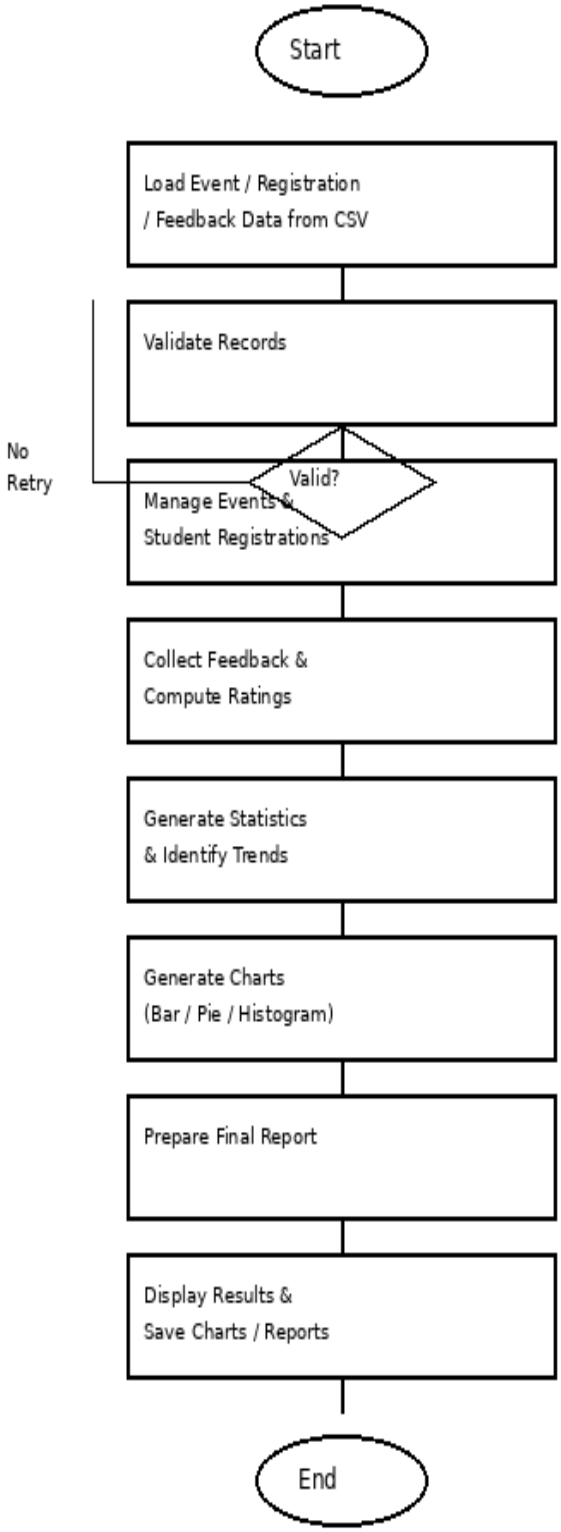


Fig 5.7: Flow Chart

CHAPTER 6: PROPOSED DESIGN

This chapter explains how the system processes event, registration, and feedback data and produces analysis results and charts. The method follows a sequential pipeline that begins with CSV input and ends with statistics and visualizations. The overall workflow is shown in the diagrams, and each stage is described in detail.

The application loads event records from events.csv, registration data from registrations.csv, and student data from students.csv, creates structured data objects, computes feedback and popularity metrics, filters important cases, and generates charts. This method allows users to obtain insights from campus event data without manual calculations or complex tools.

The process includes the following key stages:

1. Data loading from CSV files
2. Event and registration data modeling
3. Feedback and popularity analysis
4. Statistical summary generation
5. Chart creation and display
6. Final report presentation The steps included are –

Step 1: Data Loading

The program reads the events.csv, registrations.csv, and students.csv files using pandas or standard CSV handling and prepares a clean dataset for analysis.

Step 2: Event & Registration Modeling

Data structures represent individual events with attributes such as event ID, name, date, category, and capacity. Registration records link students to events. Objects or DataFrame rows are created for further processing.

Step 3: Analysis

The analysis logic performs:

- Calculation of average feedback ratings
- Registration count and popularity computation
- Identification of top-rated and low-rated events
- Event-level and overall aggregates (highest/lowest rated, most registered)

Step 4: Chart Generation

The chart functions use matplotlib to produce visual representations such as:

- Rating distribution pie/bar charts
- Event popularity histograms or bar charts
- Registration comparison plots
- Category-wise or trend views when applicable

Step 5: Main Program Flow

main.py orchestrates the process: loads data, calls analysis functions, triggers chart generation, and presents results through a simple menu or sequential execution.

Step 6: Final Output

Users receive console statistics, lists of popular or low-rated events, and generated chart images. This supports both verification and decision-making.

This methodology is effective because:

- Data processing is automated and repeatable
- Modular design makes the code easy to understand and extend
- Visualizations improve insight communication
- No external specialized software is required beyond standard Python libraries
- The workflow is suitable for undergraduate-level implementation while demonstrating practical data analysis techniques

REPRESENTATIVE CODE STRUCTURE (based on project files)

- main.py: Entry point that loads data, invokes analysis and chart functions, and handles user interaction.
- events.csv: Sample event dataset used for testing and demonstration.
- registrations.csv: Sample registration dataset.
- students.csv: Sample student dataset.

CHAPTER 7: EXPERIMENTAL RESULT

The results focus on how the CSV data was processed, how event, registration, and feedback metrics were calculated, how statistics were produced, and how charts were generated. Screenshots of console outputs, charts, and sample reports can be inserted in the respective places.

1. Data Loading Stage

The system successfully reads the events.csv, registrations.csv, and students.csv files and prepares the data for analysis.

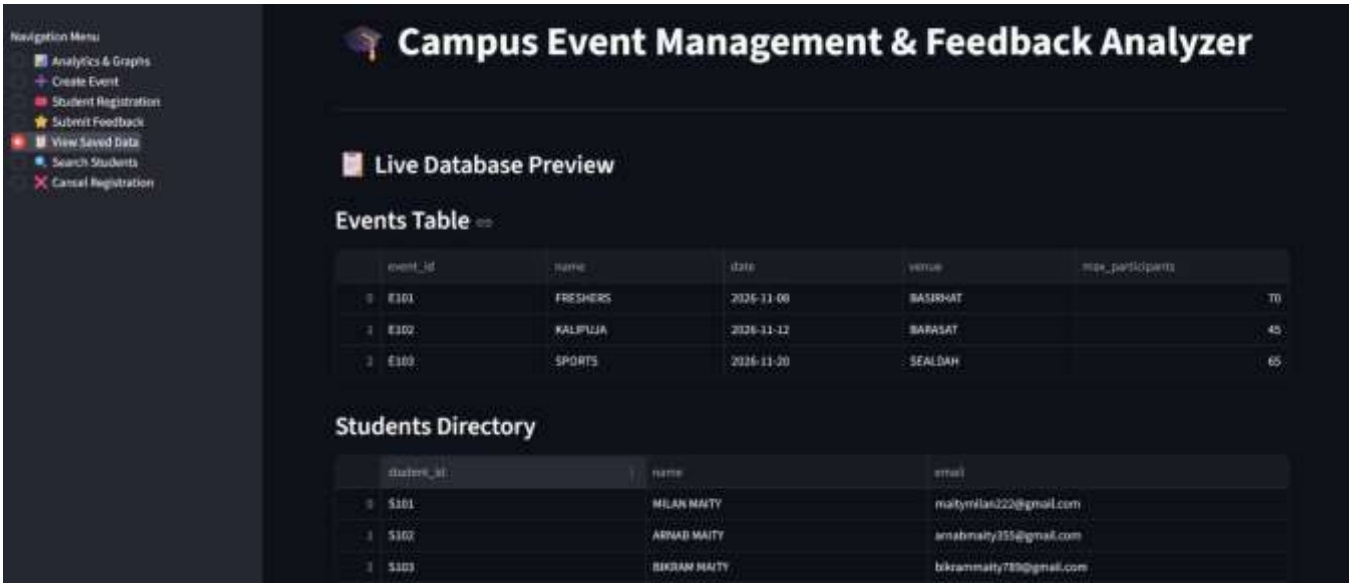


Fig 7.1: Data Loading Stage

2. Event Management & Registration

Events can be listed, added, or updated. Student registrations are recorded and counted per event.

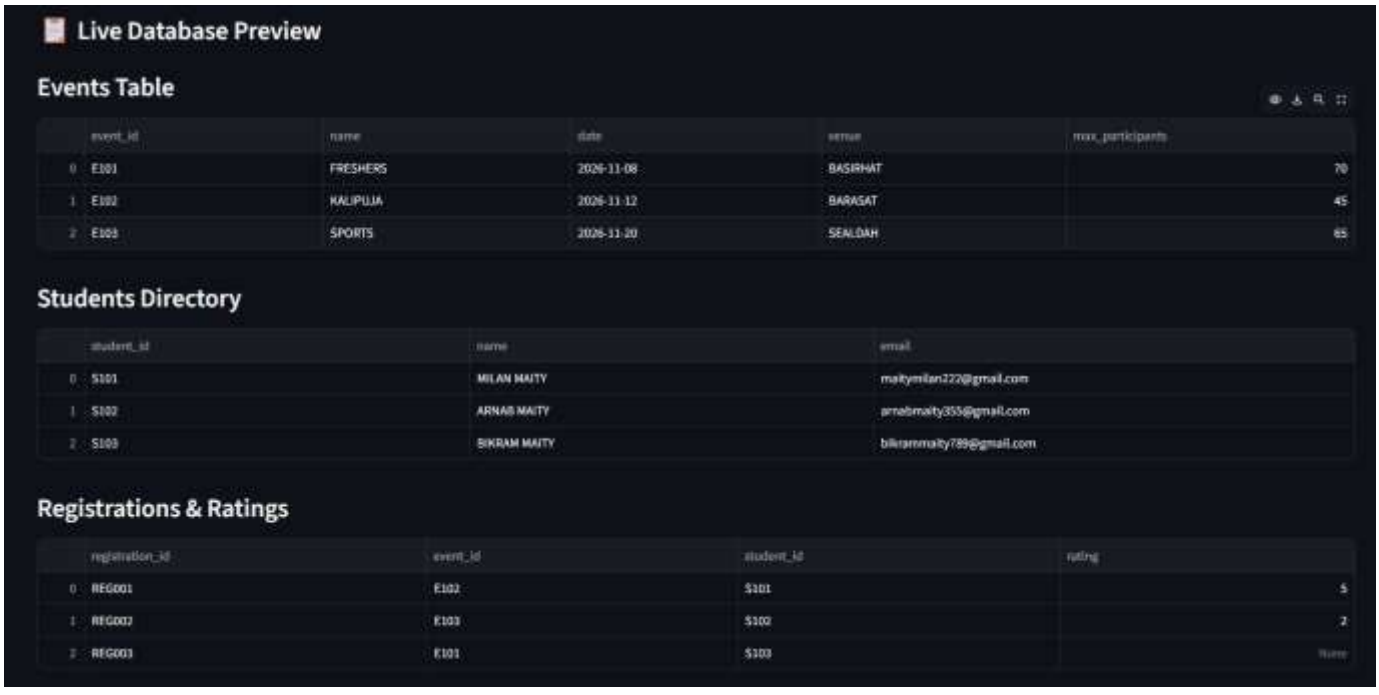


Fig 7.2: Event Management

3. Feedback Analysis

Average ratings are computed, and events falling below a rating threshold are identified.

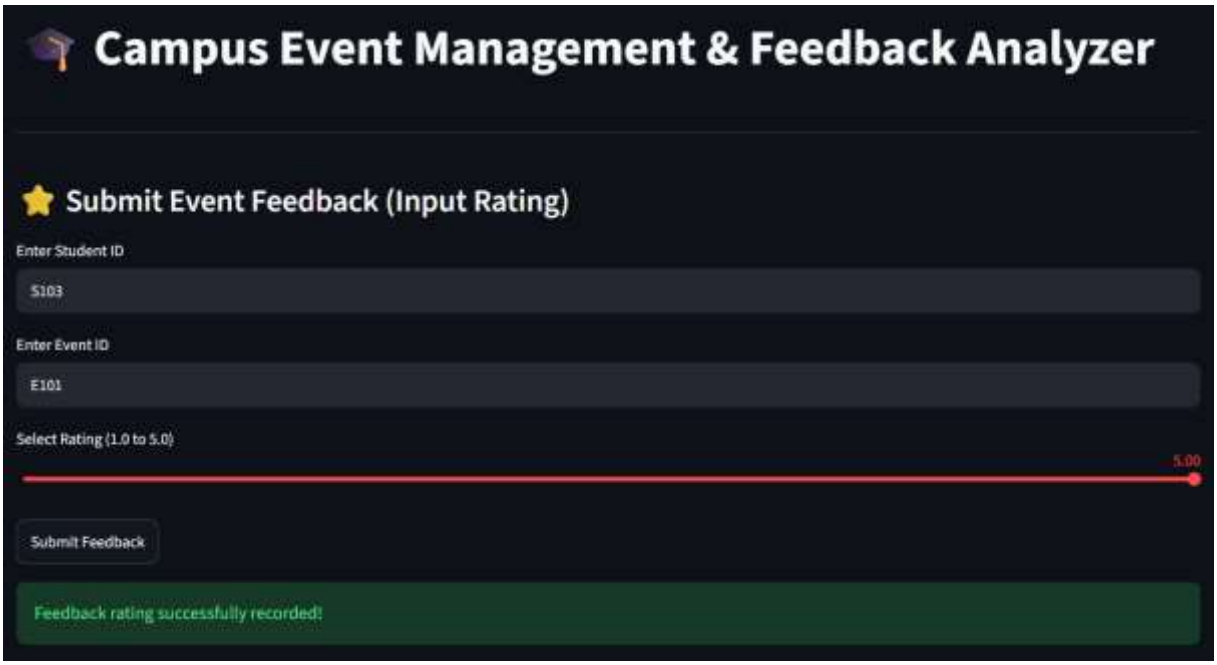


Fig 7.3: Feedback Analysis

4. Charts Generation

Visual charts (rating distribution, event popularity, registration comparison) are produced.

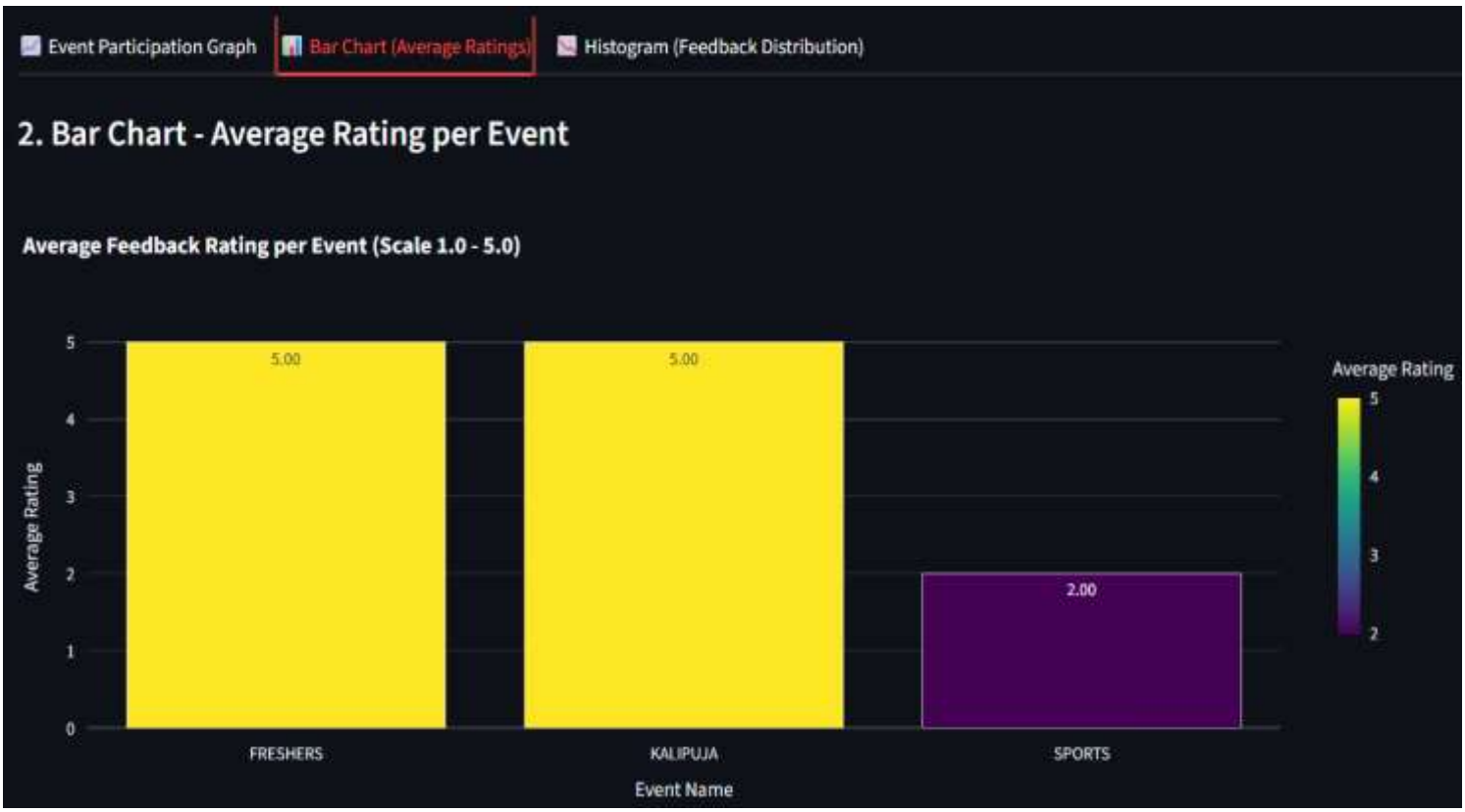


Fig 7.4: Bar Chart Generation



Fig 7.5: Histogram Generation



Fig 7.6: Event Participation Generation

CHAPTER 8: FUTURE SCOPE

Several improvements can be added in later versions of the project:

1. Enhanced User Interface

Develop a graphical user interface (Tkinter, Streamlit, or web-based dashboard) for easier data upload, interactive filtering, and real-time visualization.

2. Multi-Campus & Multi-Department Support

Extend the data model to handle multiple campuses, departments, continuous event series, and semester-wise tracking.

3. Report Export Features

Add options to export analysis results and charts to PDF, Excel, or PowerPoint for easy sharing with administrators and organizers.

4. Predictive Analytics

Incorporate simple machine learning models to predict event popularity or identify at-risk events based on historical patterns.

5. Database Integration

Replace or supplement the CSV input with a proper database (SQLite/MySQL) for larger datasets and multi-user access.

6. Notification & Alert System

Add automatic alerts or email notifications for low-rated events or registration capacity issues.

7. Advanced Visualizations

Include more sophisticated charts, heatmaps, trend lines over time, and interactive dashboards.

CHAPTER 9: CONCLUSION

The project successfully showcases a complete, end-to-end workflow for managing campus events, student registrations, and analyzing feedback from structured data through an automated and modular Python pipeline. Beginning with CSV input, the system efficiently loads records, computes meaningful metrics (average ratings, popularity, registration counts), identifies events requiring attention, and generates clear visualizations. This demonstrates that practical campus event analytics can be achieved using standard Python libraries when the process is divided into clear, modular stages.

Moreover, the modular design significantly enhances maintainability and usability, allowing individuals with basic Python knowledge to understand and extend the system. The design emphasizes accessibility and practicality, ensuring that event and feedback monitoring is not limited to complex enterprise tools. Overall, the project meets its primary objective by delivering a functional, data-driven Campus Event Management & Feedback Analyzer that is both practical and educationally valuable.

REFERENCES

1. pandas Development Team. pandas: Powerful data structures for data analysis. Available at: https://pandas.pydata.org/

2. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering. Available at: https://matplotlib.org/

3. Python Software Foundation. Python Language Reference. Available at: https://www.python.org/

4. Project source code and files. Available at: https://1drv.ms/f/c/9A2F6EF399B9BA04/IgB Fa0x_-f1jQ4smR7oUKd1yAaZ3dq6lONg1J7CKNSS4Cv4?e=WL4CwE